

X2UHJ: From AmbiX to UHJ C-Format

The Mathematics of the SEAM-LTM UHJ Plugin

Giuseppe Silvi — SEAM

2026

Abstract

This document derives the mathematics of the `x2uhj` plugin step by step. It covers the AmbiX-to-UHJ C-format encoding, the origin of Gerzon’s matrix coefficients, and the sample-rate-independent design of the quadrature all-pass network.

Contents

1	Introduction and Historical Framing	1
2	AmbiX to FuMa	2
3	The UHJ C-Format Matrix	2
4	Reconstructing Gerzon’s Coefficients	2
5	The j Problem: Wideband 90 Degrees	3
6	The Sample-Rate-Independent Design	4
7	Validation	5
A	Coefficient Tables	5
B	C++ Core	6
C	Migration Path	6

1 Introduction and Historical Framing

UHJ is a hierarchical multichannel encoding system that represents a periphonic or horizontal sound field in a small number of channels suitable for stereo-compatible transmission and subsequent decoding. The name “UHJ” derives from the BBC Matrix-H system and the 45J variant, both developed at the BBC in the 1970s as practical approaches to compatible surround sound [1]. Michael Gerzon unified and extended these systems in his 1985 AES paper [2], establishing the coefficient set that the `x2uhj` plugin implements. The C-format variant carries four channels: the stereo pair (L, R) , a third directional channel T , and a height-related channel Q .

The `x2uhj` plugin accepts a First-Order AmbiX signal (four channels in ACN order, SN3D normalisation) and delivers UHJ C-format output. This document derives Gerzon’s matrix coefficients by inverse verification against known decoded loudspeaker positions. It establishes the quadrature all-pass network as a sample-rate-independent design procedure, enabling consistent behaviour across all standard audio sample rates.

2 AmbiX to FuMa

The plugin first converts the incoming ACN/SN3D channels to the FuMa convention used throughout the UHJ literature. ACN orders the four first-order channels as A_0 (omnidirectional), A_1 (front-back), A_2 (up-down), A_3 (left-right), corresponding to the spherical harmonic sequence W, Y, Z, X . FuMa arranges the same signals as W, X, Y, Z and scales the omnidirectional component by $1/\sqrt{2}$. The conversion is therefore a reindexing together with a single rescaling:

$$W = \frac{1}{\sqrt{2}} A_0, \quad X = A_3, \quad Y = A_1, \quad Z = A_2. \quad (1)$$

All subsequent matrix operations in this document refer to the FuMa signals W, X, Y, Z .

3 The UHJ C-Format Matrix

Gerzon's UHJ encoding proceeds in two stages: formation of the intermediate sum and difference signals Σ and Δ , followed by the channel assignments for L, R, T , and Q .

$$\Sigma = 0.9396926 W + 0.1855740 X, \quad (2)$$

$$\Delta = j(-0.3420201 W + 0.5098604 X) + 0.6554516 Y, \quad (3)$$

$$L = \frac{1}{2}(\Sigma + \Delta), \quad R = \frac{1}{2}(\Sigma - \Delta), \quad (4)$$

$$T = j(-0.1432 W + 0.6512 X) - 0.7071 Y, \quad Q = 0.9772 Z. \quad (5)$$

Σ is the forward-facing sum channel; its first-order pattern secures mono compatibility across all playback configurations. Δ is the difference channel; the factor j introduces a broadband 90-degree phase shift that encodes lateral width. L and R are the two transmitted stereo channels, formed from the sum and difference of Σ and Δ . T is the third C-format channel; it carries additional directional resolution that extends spatial accuracy beyond the stereo base [3]. Q is the height-related channel; it carries the vertical component Z scaled by 0.9772 to match the overall normalisation scheme. The symbol j denotes a broadband 90-degree phase shift applied across the full audio bandwidth; its implementation as a sample-rate-independent all-pass network is the subject of Sections 5 and 6.

4 Reconstructing Gerzon's Coefficients

Gerzon chose the UHJ coefficients to reproduce the psychoacoustic localization of the original sound field. The velocity vector $\mathbf{r}_V$ governs perceived direction below roughly 700 Hz, and the energy vector $\mathbf{r}_E$ governs it above. Both vectors are weighted averages of the loudspeaker direction unit vectors $\hat{\mathbf{u}}_i$:

$$\mathbf{r}_V = \frac{\sum_i g_i \hat{\mathbf{u}}_i}{\sum_i g_i}, \quad \mathbf{r}_E = \frac{\sum_i g_i^2 \hat{\mathbf{u}}_i}{\sum_i g_i^2}. \quad (6)$$

The velocity vector uses the loudspeaker gains g_i directly, and the energy vector uses their squared values. A well-designed encoding keeps both vectors pointing toward the original source azimuth, with magnitude near unity.

The constants in the Σ and Δ expressions carry geometric meaning. The coefficient $0.9396926 = \cos 20^\circ$ and $0.3420201 = \sin 20^\circ$ reveal the 20° design angle that Gerzon selected to balance spatial precision and robust mono compatibility. Σ realises a forward first-order microphone pattern, which secures mono compatibility across all playback configurations. The j term in Δ supplies the degree of freedom that maps the full 360° of azimuth into two channels reversibly.

Inverse verification takes the published coefficients as given and confirms numerically that they realise good localization. The verification evaluates two listening models. The surround model feeds (L, R) to a symmetric illustrative loudspeaker layout and reads the localization vectors of the reconstructed field. The super-stereo model feeds (L, R) to two frontal loudspeakers at 30° and -30° and reads the localization a listener perceives at those positions.

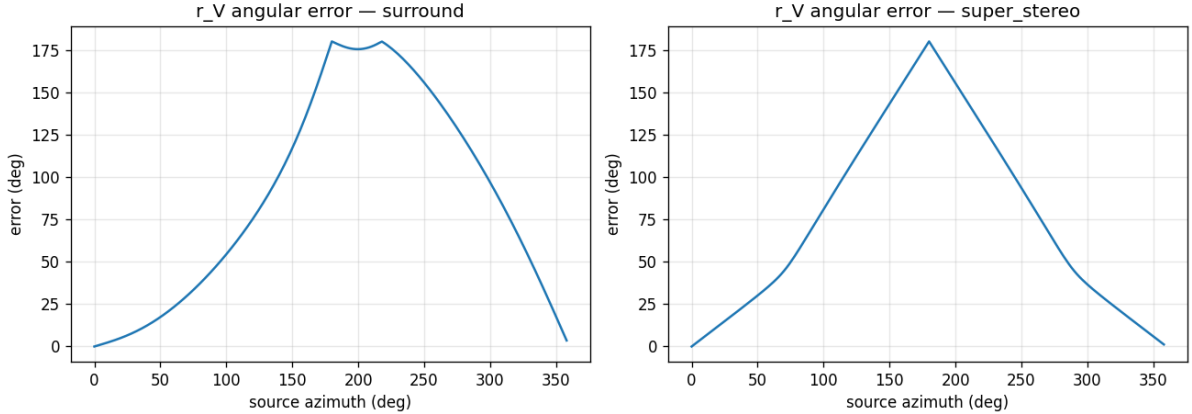


Figure 1: Velocity-vector angular error versus source azimuth: surround decode (left) and super-stereo (right). Both models confirm that Gerzon’s published coefficients localise the source across azimuth.

The surround model illuminates why Gerzon selected these coefficients: the reconstructed field localises faithfully across the full azimuth circle. The super-stereo model demonstrates the stereo listening result: a frontal pair at $\pm 30^\circ$ conveys the same source direction with small angular error. Agreement between the two models across azimuth confirms that the published coefficients serve both listening contexts.

5 The j Problem: Wideband 90 Degrees

The j operator in the UHJ matrix denotes a 90-degree phase shift applied uniformly across the audio band. A broadband 90-degree phase difference between two filter paths realises this operator. A FIR Hilbert pair achieves the phase difference with a tap set fixed to one sample rate and one length. An IIR all-pass pair carries the same phase relation in physical (f_0, Q) units, which expresses the design in sample-rate-free terms [4, 5]. This section develops the IIR all-pass approach and establishes the procedure that holds the quadrature at every sample rate.

The classic realisation of the broadband 90-degree pair is a two-path polyphase network of first-order all-pass sections, designed in normalised frequency [6, 7, 8]. A widely used audio design holds the difference within 0.7° across most of the band with eight multiplies [8]. The polyphase first-order structure is well documented by half-band and Hilbert transformer theory [9], and the open C++ library HIIR provides a production-ready realisation [10]. The open WigWare UHJ encoder follows this polyphase form with fixed normalised coefficients [11], so its protected frequency band scales with the sample rate. The Ambisonic Toolkit takes the convolution route with a single impulse response that it applies at every sample rate [12]. Minimax and least-squares all-pass phase design [13] and the half-band-to-Hilbert procedure [14] supply the methods this document applies. The normalised-frequency design tradition reaches back to the analog phase-difference networks [15].

6 The Sample-Rate-Independent Design

The analog second-order all-pass prototype and its phase response are:

$$H_a(s) = \frac{s^2 - (\omega_0/Q)s + \omega_0^2}{s^2 + (\omega_0/Q)s + \omega_0^2}, \quad \omega_0 = 2\pi f_0. \quad (7)$$

$$\varphi_a(\omega) = -2 \arctan \frac{(\omega_0/Q)\omega}{\omega_0^2 - \omega^2}. \quad (8)$$

Each network is a cascade of three second-order all-pass sections, one for the real path H_R and one for the imaginary path H_I . Optimising the six (f_0, Q) pairs so that $\varphi_{H_I} - \varphi_{H_R} = -90^\circ$ over 20 Hz to 20 000 Hz on the analog phase reaches a maximum error of 0.40° in the continuous domain.

Table 1: Analog s -domain ideal (f_0, Q) pairs. Maximum phase error 0.40° across 20 Hz to 20 000 Hz.

Path	f_0 (Hz)	Q
H_R	107.988	0.2267
H_R	449.085	0.2406
H_R	19474.604	0.3194
H_I	20.540	0.3194
H_I	1756.151	0.4026
H_I	1878.679	0.1198

The RBJ bilinear realisation maps each analog section to a digital biquad all-pass [16]:

$$a_1 = \frac{-2 \cos \omega}{1 + \alpha}, \quad a_2 = \frac{1 - \alpha}{1 + \alpha}, \quad \alpha = \frac{\sin \omega}{2Q}, \quad \omega = \frac{2\pi f_0}{f_s}, \quad (9)$$

with all-pass numerator $b = [a_2, a_1, 1]$.

The bilinear transform maps a digital frequency f to the analog frequency $F = (f_s/\pi) \tan(\pi f/f_s)$. A fixed (f_0, Q) set therefore presents its broadband phase on a warped frequency axis that depends on f_s . The quadrature holds near the sample rate the set was designed for, and the phase difference departs from 90° as the sample rate moves away. Sample-rate independence belongs to the design procedure: re-running the digital phase fit at the actual sample rate restores the quadrature at that rate. The deliverable stores a per-rate (f_0, Q) table, with one entry produced by the same procedure for each standard rate.

Table 2: Per-rate design maxima. Each row is designed independently at its own sample rate.

Sample rate (kHz)	Max phase error (deg)
44.1	2.040
48	1.362
88.2	0.529
96	0.505
176.4	0.429
192	0.425

The per-rate table holds the quadrature within roughly two degrees at every standard rate. Section 7 visualises this against the phase drift of a fixed set applied across varying sample rates.

7 Validation

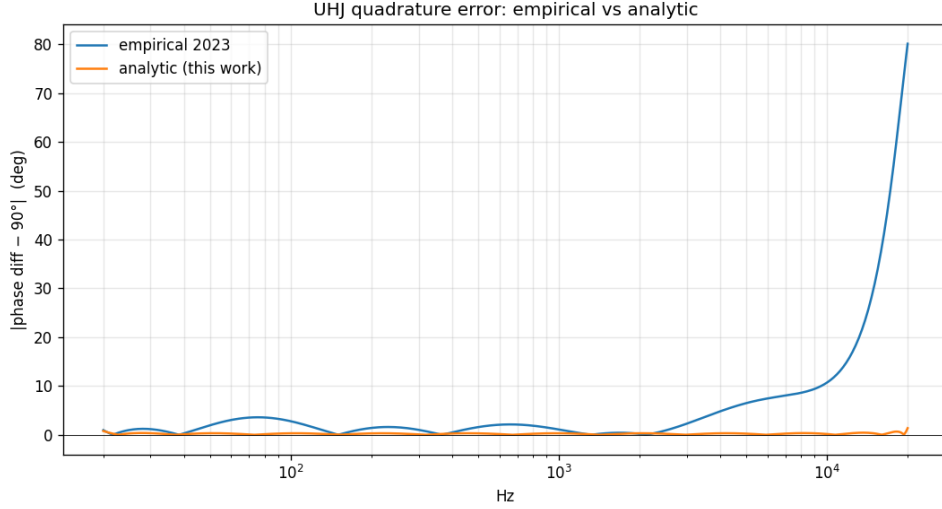


Figure 2: Quadrature phase error across frequency: the 2023 empirical design and the analytic design.

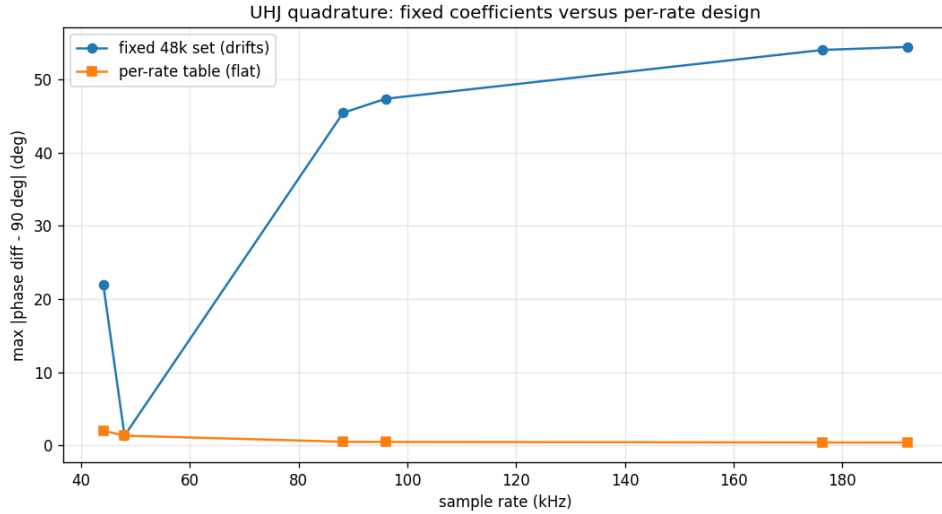


Figure 3: Maximum quadrature error versus sample rate: the fixed 48 kHz coefficient set drifts, and the per-rate table stays flat.

The fixed coefficient set holds the quadrature near 48 kHz, achieving 1.36° maximum error at that rate [4]. The per-rate table holds the error within roughly two degrees at every standard rate.

A Coefficient Tables

Table 4 lists the analog s -domain ideal (f_0, Q) pairs produced by the continuous-domain optimisation; these anchor the design before any digital realisation.

Table 5 lists the shipped 48 kHz digital (f_0, Q) pairs stored in `coeffs.json`; these are the values the plugin loads at 48 kHz.

Table 3: Maximum quadrature phase error (degrees) at each standard sample rate. The fixed 48 kHz coefficient set drifts away from 90° as the sample rate moves from 48 kHz, while the per-rate table holds the error flat across all rates.

Sample rate	Fixed 48 kHz set (deg)	Per-rate table (deg)
44.1 kHz	21.95	2.04
48 kHz	1.36	1.36
88.2 kHz	45.43	0.53
96 kHz	47.35	0.51
176.4 kHz	54.01	0.43
192 kHz	54.42	0.43

Table 4: Analog s -domain ideal (f_0, Q) pairs. Maximum phase error 0.40° across 20 Hz to 20 000 Hz.

Path	f_0 (Hz)	Q
H_R	107.988	0.2267
H_R	449.085	0.2406
H_R	19474.604	0.3194
H_I	20.540	0.3194
H_I	1756.151	0.4026
H_I	1878.679	0.1198

B C++ Core

The following listing is the realisation that consumes the per-rate (f_0, Q) values: the `AllpassSection::set` method computes the RBJ biquad coefficients from a frequency and quality factor at a given sample rate, and `AllpassSection::process` applies the difference equation one sample at a time.

```
void set(double f, double Q, double fs) {
    const double w = 2.0 * M_PI * f / fs;
    const double alpha = std::sin(w) / (2.0 * Q);
    const double n = 1.0 + alpha;
    a1 = -2.0 * std::cos(w) / n;
    a2 = (1.0 - alpha) / n;
}

double process(double x) {
    const double y = a2 * x + a1 * x1 + 1.0 * x2 - a1 * y1 - a2 * y2;
    x2 = x1; x1 = x; y2 = y1; y1 = y;
    return y;
}
```

C Migration Path

A future plugin selects the per-rate (f_0, Q) entry by sample rate from the generated table `coeffs_perfs.json`. For a sample rate outside the standard set, the plugin selects the nearest table entry. A closed-form prewarp-remap of one reference design serves as an approximation; it holds within roughly one degree for sample rates at and above 48 kHz, and it reaches about 12° at 44.1 kHz. The plugin C++ change that consumes the table is a separate follow-up with its own specification and audio testing.

Table 5: Shipped 48 kHz digital (f_0, Q) pairs from `coeffs.json`. Maximum phase error 1.36°.

Path	f_0 (Hz)	Q
H_R	141.887	0.2019
H_R	671.746	0.2122
H_R	18654.238	0.3031
H_I	24.004	0.3090
H_I	2991.937	0.3848
H_I	3219.986	0.0963

References

- [1] M. A. Gerzon, “Non-rotationally-symmetric encoding (uhj),” Patent US4 095 049, 1977.
- [2] —, “Ambisonics in multichannel broadcasting and video,” *Journal of the Audio Engineering Society*, vol. 33, no. 11, pp. 859–871, 1985, aES Preprint 2034, 74th Convention, 1983.
- [3] “The ambisonics c-format for super stereo: an open-source decoder,” in *NYCEMF*, 2023.
- [4] A. Mastrorillo, G. Silvi, and F. Scagliola, “Implementing uhj stereo in the envelop for live suite,” 2023.
- [5] “A guide to the implementation of ambisonics super stereo.”
- [6] R. Ansari, “IIR discrete-time hilbert transformers,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 35, no. 8, pp. 1116–1119, 1987.
- [7] P. A. Regalia, S. K. Mitra, and P. P. Vaidyanathan, “The digital all-pass filter: A versatile signal processing building block,” *Proceedings of the IEEE*, vol. 76, no. 1, pp. 19–37, 1988.
- [8] O. Niemitalo, “90 degree phase difference IIR allpass pair,” <https://yehar.com/blog/?p=368>, 2003.
- [9] H. W. Schüßler and P. Steffen, “Halfband filters and hilbert transformers,” *Circuits, Systems and Signal Processing*, vol. 17, no. 2, pp. 137–164, 1998.
- [10] L. de Soras, “HIIR: An oversampling and hilbert transform library in C++,” <https://lidesoras.fr/prod.html>.
- [11] B. Wiggins, “WigWare UHJ encoder/decoder (Reaper JSFX),” <https://www.brucewiggins.co.uk/>.
- [12] J. Anderson and Ambisonic Toolkit Community, “Ambisonic toolkit: UHJ kernels,” <https://github.com/ambisonictoolkit/atk-kernels>.
- [13] M. Lang, “Allpass filter design and applications,” *IEEE Transactions on Signal Processing*, vol. 46, no. 9, pp. 2505–2514, 1998.
- [14] D. Harris, E. Berdahl, and J. S. Abel, “An infinite impulse response (IIR) hilbert transformer filter design technique for audio,” in *Proc. 129th Audio Engineering Society Convention*, 2010, paper 8258.
- [15] S. D. Bedrosian, “Normalized design of 90-degree phase-difference networks,” *IRE Transactions on Circuit Theory*, vol. 7, no. 2, pp. 128–136, 1960.

- [16] R. Bristow-Johnson, “Cookbook formulae for audio EQ biquad filter coefficients,” Audio EQ Cookbook.